Học Sâu (Deep Learning)

Chương 12: Phát hiện Vật thể (Object Detection)

Đại học Đông Á

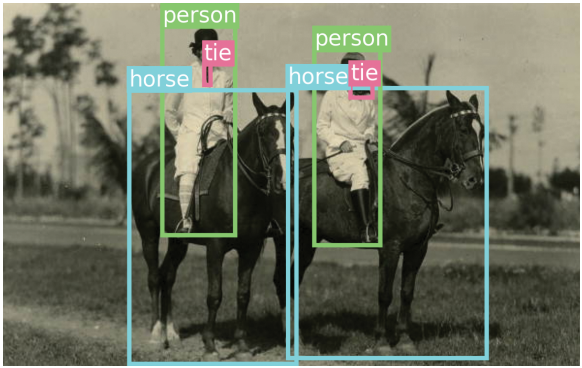
August 14, 2026

Nội dung chương

1. 1. Giới thiệu Bài toán Phát hiện Vật thể
2. 2. Kiến trúc Hai giai đoạn (Two-stage Detectors)
3. 3. Kiến trúc Một giai đoạn (Single-stage Detectors)
4. 4. Thuật toán YOLO (You Only Look Once)
5. Tổng kết

Bài toán Phát hiện Vật thể là gì?

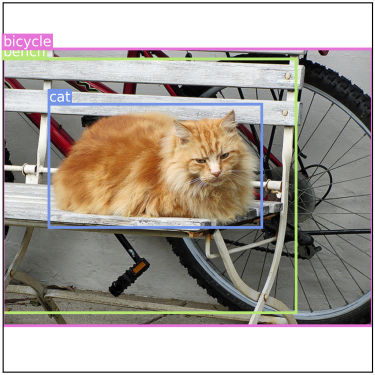
- Không chỉ trả lời câu hỏi "Trong ảnh có gì?" (Classification).
- Phải trả lời được "Nó nằm ở đâu?" bằng cách vẽ các **Hộp giới hạn (Bounding Boxes)**.
- **Ứng dụng thực tế:**
  - Xe tự lái: Phát hiện người đi bộ, biển báo.
  - Bán lẻ: Đếm số lượng sản phẩm trên kệ.
  - Trích xuất: Cắt vùng biển số xe để đưa vào OCR.



Hình 12.1: Vẽ hộp giới hạn và gán nhãn

Bộ dữ liệu COCO (Common Objects in Context)

- MS COCO là tiêu chuẩn vàng (Gold Standard) để đánh giá các mô hình Object Detection hiện đại.
- Hơn 200,000 hình ảnh được gán nhãn thủ công với 80 danh mục vật thể.
- Tính chất đa dạng cao: Nhiều vật thể đan xen, chồng chéo, kích thước từ siêu nhỏ đến khổng lồ trong cùng một khung hình.



Hình 12.2: Dữ liệu phức tạp trong COCO

Tải Dữ liệu COCO 2017

Dữ liệu COCO có dung lượng khổng lồ. Chúng ta tải bộ `train2017` (khoảng 18GB) và file nhãn (Annotations JSON).

```
import keras
import keras_hub

images_path = keras.utils.get_file(
    "coco",
    "http://images.cocodataset.org/zips/train2017.zip",
    extract=True,
)
annotations_path = keras.utils.get_file(
    "annotations",
    "http://images.cocodataset.org/annotations/annotations_trainval2017.zip",
    extract=True,
)
```

Xử lý Tọa độ Hộp giới hạn (Bounding Boxes)

Mỗi hộp trong COCO có tọa độ `x, y, width, height`. Chúng ta cần chuẩn hóa tọa độ này về tỷ lệ `[0, 1]` dựa trên kích thước ảnh gốc.

```python
import json

with open(f"{annotations_path}/annotations/instances_train2017.json", "r") as f:
    annotations = json.load(f)

# Sorts image metadata by ID
images = {image["id"]: image for image in annotations["images"]}

# Converts bounding box to coordinates on a unit square
def scale_box(box, width, height):
    scale = 1.0 / max(width, height)
    x, y, w, h = [v * scale for v in box]
    x += (height - width) * scale / 2 if height > width else 0
    y += (width - height) * scale / 2 if width > height else 0
    return [x, y, w, h]
```

Tích hợp Dữ liệu Meta

Nhóm tất cả các Bounding Boxes và Labels theo từng file hình ảnh.

```
# Aggregates all bounding box annotations by image ID
metadata = {}
for annotation in annotations["annotations"]:
    id = annotation["image_id"]
    if id not in metadata:
        metadata[id] = {"boxes": [], "labels": []}
    image = images[id]
    box = scale_box(annotation["bbox"], image["width"], image["height"])
    metadata[id]["boxes"].append(box)
    metadata[id]["labels"].append(annotation["category_id"])
    metadata[id]["path"] = images_path + "/train2017/" + image["file_name"]

metadata = list(metadata.values())
```

Kết quả ta thu được $117,266$ ảnh huấn luyện!

Hàm Hỗ trợ Trực quan hóa (Visualization Utilities)

Để dễ dàng theo dõi Bounding Box trên ảnh, ta viết các hàm phụ trợ dùng `matplotlib`.

```python
import matplotlib.pyplot as plt
from matplotlib.colors import hsv_to_rgb
from matplotlib.patches import Rectangle

color_map = {0: "gray"}

def label_to_color(label):
    if label not in color_map:
        h, s, v = (len(color_map) * 0.618) % 1, 0.5, 0.9
        color_map[label] = hsv_to_rgb((h, s, v))
    return color_map[label]

def draw_box(ax, box, text, color):
    x, y, w, h = box
    ax.add_patch(Rectangle((x, y), w, h, lw=2, ec=color, fc="none"))
    textbox = dict(fc=color, pad=1, ec="none")
    ax.text(x, y, text, c="white", size=10, va="bottom", bbox=textbox)
```

Lọc và Shuffle Dữ liệu

Chỉ giữ lại các ảnh chứa tối đa 4 vật thể để đơn giản hóa quá trình tính toán trong giới hạn bài học.

```
import random

# Limits the dataset to images with 4 or fewer boxes
metadata = list(filter(lambda x: len(x["boxes"]) <= 4, metadata))

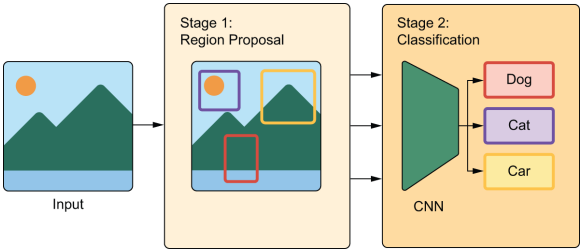
# Shuffles data to prevent grouping by object type
random.shuffle(metadata)
```

Họ mô hình R-CNN (Region-based CNN)

Kiến trúc Hai giai đoạn ưu tiên **Độ chính xác cao**:

1. **Giai đoạn 1 (Đề xuất vùng - Region Proposals):** Quét toàn bộ ảnh để tìm ra vài ngàn Vùng (Regions) "có vẻ chứa vật thể".
2. **Giai đoạn 2 (Phân loại):** Đưa từng vùng đó qua mạng CNN để dự đoán Nhãn (Label) và Tinh chỉnh tọa độ hộp (Box Refinement).

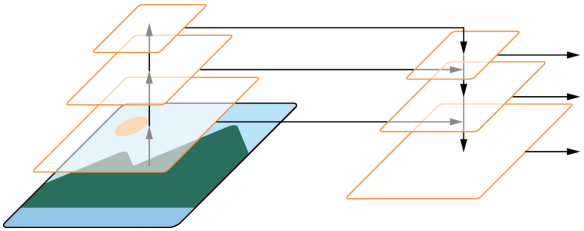
Nhược điểm: Rất chậm, không thể chạy Real-time.



Hình 12.3: Quy trình xử lý của R-CNN

Kiến trúc Single-stage và FPN

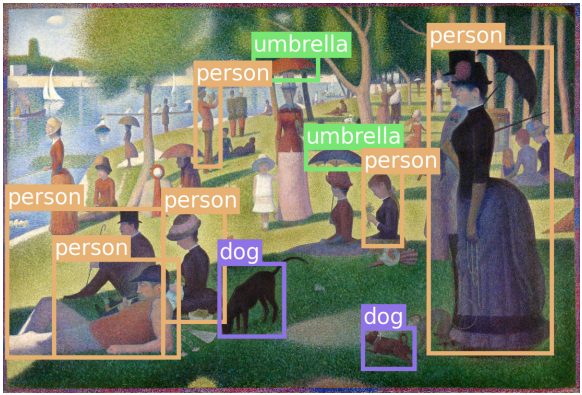
- Dự đoán song song Tọa độ hộp và Nhãn ngay trong 1 bước duy nhất (YOLO, RetinaNet, SSD).
- **Vấn đề:** Vật thể nhỏ dễ bị mất dấu ở các lớp sâu.
- **Giải pháp FPN (Feature Pyramid Network):** Hợp nhất đặc trưng ở nhiều độ phân giải khác nhau, giúp mạng phát hiện được vật thể ở mọi kích cỡ (Từ nhỏ xíu đến to lớn).



Hình 12.4: Mạng tháp đặc trưng (FPN)

Kết quả dự đoán của RetinaNet

- RetinaNet là một trong những Single-stage Detector xuất sắc nhất của Facebook AI.
- Sử dụng hàm **Focal Loss** để giải quyết bài toán mất cân bằng lớp trầm trọng giữa Tiền cảnh (Chứa vật thể) và Hậu cảnh (Không chứa gì).
- Dự đoán mượt mà, bao quát toàn bộ khung hình một cách chính xác.



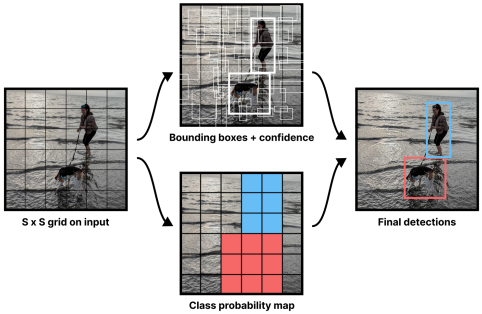
Hình 12.5: Dự đoán của RetinaNet

Kỷ nguyên mới mang tên YOLO

- Được giới thiệu năm 2015 bởi Joseph Redmon, YOLO làm rung chuyển giới AI.
- Chuyển bài toán Detection thành **Regression (Hồi quy) trên lưới (Grid)**.
- Khả năng chạy Real-time lên tới 45-155 FPS trên GPU, vượt xa hoàn toàn R-CNN.
- Nguyên lý: Chia ảnh thành lưới $S \times S$ (VD: 6×6). Mỗi ô lưới (Cell) chịu trách nhiệm dự đoán các vật thể có tâm (Center) nằm trong ô đó.

Tư duy Chia lưới và Anchor Boxes

- Hình bên trái: Chia ảnh gốc thành Lưới (Grid). Ô màu cam chứa Tâm của 1 con chó. Do đó ô lưới màu cam phải chịu trách nhiệm dự đoán con chó này.
- Hình bên phải: Mô hình sẽ không dự đoán tọa độ ngẫu nhiên. Nó sẽ xuất ra các đặc trưng Confidence, Bounding Box và Label cùng lúc.



Hình 12.6: Cơ chế hoạt động của YOLO

Nạp Mạng Xương sống (Backbone)

Thay vì đào tạo từ đầu, ta sử dụng mạng `ResNet50` đã được huấn luyện sẵn trên ImageNet từ `KerasHub` để đóng vai trò làm Bộ trích xuất Đặc trưng (Feature Extractor).

```
image_size = 448

# Load ResNet50 pretrained backbone
backbone = keras_hub.models.Backbone.from_preset(
    "resnet_50_imagenet",
)

# Setup the preprocessor for input resizing
preprocessor = keras_hub.layers.ImageConverter.from_preset(
    "resnet_50_imagenet",
    image_size=(image_size, image_size),
)
```